VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING




# CO5177 ASSIGNMENTS REPORT

---

## PROGRAMMING FOUNDATION FOR DATA ANALYTICS AND VISUALIZATION

***Group: Trailblazer***

---

**Instructor**: Dr. Le Thanh Sach

**Student**: Dinh Hoang Duy Khanh 2670306

*Ho Chi Minh City, November 2026*

# Member List and Workload

| No. | Name | Student ID | Main responsibilities | Contribution |
|---|---|---|---|---|
| 1 | Dinh Hoang Duy Khanh | 2670306 | Build the Tabular, Text, and Image EDA notebooks, train and evaluate models, create the website and write the report. | 100% |

# Contents

**Member List and Workload** **2**

**Introduction** **5**

**1 Tabular Data: Kepler Objects of Interest** **8**
1.1 Exploratory Data Analysis . . . 8
1.1.1 Dataset Size and Target Distribution . . . 8
1.1.2 Data Quality, Missing Values, and Leakage Control . . . 9
1.1.3 Numerical Distributions and Outliers . . . 10
1.1.4 Feature Distributions by KOI Disposition . . . 13
1.2 Preprocessing and Evaluation Design . . . 13
1.2.1 Feature Preparation . . . 13
1.2.2 Group-Aware Train, Validation, and Test Split . . . 14
1.3 Model Comparison and Final Evaluation . . . 15
1.3.1 Baseline and Balanced Random Forest . . . 15
1.3.2 Validation Model Comparison . . . 15
1.3.3 Final Test Results and Feature Importance . . . 16
1.3.4 Key Findings and Limitations . . . 17

**2 Text Data: ViCTSD Vietnamese Comments** **18**
2.1 Exploratory Data Analysis . . . 18
2.1.1 Dataset Size, Official Split, and Labels . . . 18
2.1.2 Topic Balance and Comment Length . . . 19
2.1.3 Vietnamese Word-Count Distribution . . . 19
2.2 Text Preparation and Leakage Control . . . 20
2.2.1 Simple Normalization . . . 20
2.2.2 Duplicate and Split-Overlap Check . . . 21
2.3 Feature Representation . . . 21
2.3.1 Word and Character TF-IDF . . . 21
2.4 Toxicity Model Comparison . . . 22
2.4.1 Validation Models and Selection . . . 22
2.5 Final Test Results and Error Review . . . 23
2.5.1 Test Metrics and Confusion Matrices . . . 23
2.5.2 Examples of Classification Errors . . . 24
2.5.3 Key Findings and Limitations . . . 25

**3 Image Data: Intel Image Classification** **26**
3.1 Task Objective and Dataset . . . 26
3.2 Dataset Structure and Image EDA . . . 26
3.3 Split, Quality Control, and Leakage Prevention . . . 28
3.4 Preprocessing and Augmentation . . . 28
3.5 Model Comparison . . . 29
3.6 Final Test Evaluation and Error Review . . . 30

**4 Overall Conclusion** **33**
4.1 Summary of Final Results . . . 33
4.2 Limitations and Future Work . . . 33

**Reproducibility** **34**

# Introduction

This report presents three completed data-analysis tasks for CO5177. The first task uses NASA Kepler Objects of Interest (KOI) data in tabular form. The second task uses ViCTSD, a Vietnamese comment dataset. The third task uses Intel Image Classification, a labelled natural-scene image dataset. All three tasks follow the same sequence: inspect the data, check quality, prepare inputs, train simple models, and evaluate on a held-out test set.

The report only states results produced by the accompanying notebooks. The goal is not to claim that the models solve an astronomy or moderation problem by themselves. They are reproducible learning exercises that show the effect of EDA and careful evaluation.

For a reader new to this topic, exploratory data analysis (EDA) is the step taken before model training. It answers practical questions: What does one row mean? Which columns are incomplete? Are some classes much more common than others? Are there values that are unusually large? The answers determine what can safely be given to a model. EDA is not only a collection of plots. Each plot is used to make one clear decision about cleaning, feature preparation, or evaluation.

All three tasks are supervised classification tasks. This means that the dataset already contains an answer for past observations, called the target label. The model learns from labelled training examples, then predicts labels for comments, catalogue objects, or images it has not seen before. To make this test fair, the validation and test data must be kept separate from the training data. The validation split is used to compare simple model choices. The test split is kept for one final check after a model has been selected.

## Assignment Context and Objective

This individual submission follows the course-project structure: one required Tabular task and two additional tasks for Text and Image data[1]. The KOI data satisfies the Tabular requirements because it has more than 2,000 rows, more than 10 columns, missing values, numerical and categorical fields, class imbalance, and outliers. ViCTSD supplies more than 2,000 Vietnamese text samples, which fits the recommended Text-data direction. Intel Image Classification supplies 17,034 RGB images in six classes, which is above the assignment requirement of 5,000 images and three classes for an Image task.

The three datasets are deliberately different. KOI is a table of scientific measurements, so each selected feature has a fixed column and a numerical scale. ViCTSD is a collection of short Vietnamese comments, so the useful information is written in words and character patterns. Intel Image Classification is a collection of RGB scene images, so useful information is distributed across neighbouring pixels, texture, and spatial layout. This contrast is useful for the assignment because it shows that the same Python workflow can be reused, while the representation given to a model must change.

The purpose is to demonstrate how the analytical toolkit must be adapted to the unique

properties of each data type. The same final goal—a reliable classification result—does not mean that the same preparation or visualization method can be used for every input.

## Adapting the Analytical Toolkit

For KOI, each observation is a row and each measurement is a column. The analysis therefore uses data types, missing-value counts, numerical distributions, class-conditional plots, outlier checks, correlations, encoding, and scaling. The model receives a feature matrix after the data has been transformed.

For ViCTSD, the main input is a comment, not a fixed set of numerical columns. The analysis first checks the label balance, topic distribution, comment length, and duplicate text. Text normalization is then followed by word or character TF-IDF, which converts comments into a sparse numerical matrix. N-gram weights, confusion matrices, and examples of incorrect predictions are more useful than a correlation heatmap for this task.

For Intel Image Classification, the main input is a 150 by 150 RGB scene image. The analysis uses a grid of examples, class counts, a simple brightness check, a split table, and an augmentation preview. The model receives image pixels after resizing and ImageNet normalization. A pretrained convolutional network creates a fixed feature vector, which is then classified by a simple model. This is different from tabular scaling and text TF-IDF, but it follows the same safety rule: input preparation is defined using training data and the source-test images are not used to select the model.

## Analysis Framework

The work uses four connected stages. First, define the prediction target, the dataset constraints, and fields that must not enter the model. Second, perform EDA to identify missing values, imbalance, extreme values, duplicates, or direct leakage. Third, fit every preprocessing operation on training data only and create train, validation, and test sets. Fourth, compare simple baselines with an improved model using accuracy, balanced accuracy, macro F1, confusion matrices, and feature or error analysis. The test split is used once for the selected model.

The order of these stages matters. For example, a median value must be calculated from training rows, not from the full dataset; otherwise information from validation or test rows has already influenced training. The same rule applies to a text vocabulary. The TF-IDF vectorizer learns its vocabulary from training comments, then only transforms validation and test comments. This small discipline makes the reported score closer to the score expected on new data.

## Computational Framework

The computational workflow is designed for Google Colab and Python. Pandas and NumPy load, inspect, and transform the datasets. Matplotlib and Seaborn generate the independent EDA figures. Scikit-learn supplies the preprocessing pipelines, TF-IDF vectorizers, classification models, data splits, and evaluation metrics. PyTorch and Torchvision load pretrained MobileNetV3 weights and process the Intel scene images. Each data type is organized as its own reproducible notebook workflow: load data, run EDA, prepare features, train models, export figures, and report the final metrics. The website, notebooks, figures, and PDF are kept together in the project repository so the results can be checked again.

**Scope of this report**

- **Tabular data:** 9,564 Kepler catalogue objects with three dispositions: CONFIRMED, CANDIDATE, and FALSE POSITIVE.
- **Text data:** 10,000 Vietnamese comments with two binary labels: Toxicity and Constructiveness.
- **Image data:** 17,034 Intel RGB scene images with 6 labels.
- **Reproducibility:** source data snapshots, executed notebooks, figures, the website, and PDF reports are kept in the project repository.

# 1 Tabular Data: Kepler Objects of Interest

## 1.1 Exploratory Data Analysis

### 1.1.1 Dataset Size and Target Distribution

The tabular task uses the cumulative Kepler Objects of Interest catalogue from the NASA Exoplanet Archive[2]. Each row describes a transit-like signal observed around a star. The target is `koi_disposition`, which has three classes.

The target is a catalogue decision, not a direct measurement. `CONFIRMED` means that the signal has enough supporting evidence to be treated as a confirmed planet in the catalogue. `CANDIDATE` means that the signal remains under consideration. `FALSE POSITIVE` means that the signal is explained by another cause. The model therefore learns to reproduce a catalogue label from selected measurements; it does not perform physical confirmation of a planet.

One row is one KOI object and one column is one recorded property, such as orbital period, estimated radius, transit depth, or the signal-to-noise ratio. This row-and-column structure is why a tabular workflow begins with data types, missing-value counts, and numerical distributions. Table 1.1 gives the size of the working table after the selected columns have been loaded for EDA.

Table 1.1: Kepler KOI dataset summary.

| Item | Value |
|---|---|
| Rows | 9,564 |
| Columns kept for EDA | 30 |
| Numerical columns | 25 |
| Categorical columns | 5 |
| Missing cells | 13,878 |
| Exact duplicate rows | 0 |
| Target classes | 3 |

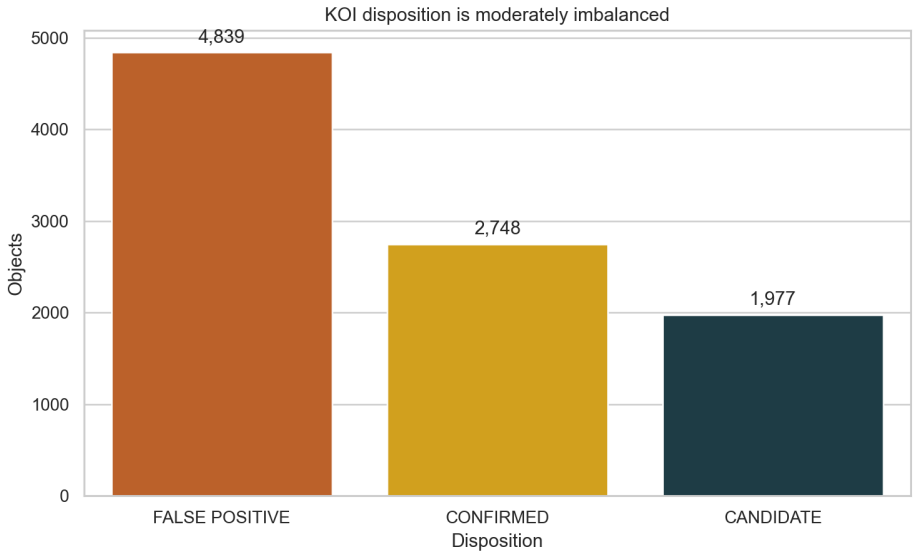



Figure 1.1: KOI disposition distribution. FALSE POSITIVE is the largest class, so accuracy is not the only evaluation metric.

The target counts are 4,839 FALSE POSITIVE objects, 2,748 CONFIRMED objects, and 1,977 CANDIDATE objects. This is a moderate class imbalance. Therefore, the report uses accuracy, balanced accuracy, and macro F1.

Accuracy is the proportion of all correct predictions. It is easy to understand, but it can look high when a model mainly predicts the largest class. Balanced accuracy first calculates recall for each class and then averages those recalls, so a rare class has the same influence as a common class. Macro F1 also gives every class equal weight by averaging its F1 score. F1 combines precision, meaning that a predicted class is correct, and recall, meaning that the model finds the true objects in that class. Using all three numbers gives a more complete view than accuracy alone.

### 1.1.2 Data Quality, Missing Values, and Leakage Control

The data contains missing values in several fields. Some fields are not suitable for prediction because they directly reveal the final decision. For example, the pipeline disposition, confidence score, official Kepler name, and false-positive flags are excluded. Using them would make the test score look better without giving a realistic predictive model.

Missing values mean that a measurement was not available for every catalogue object. A missing value is not automatically an error: a parameter may be absent because it could not be estimated or was not reported in the same way for every object. Figure 1.2 ranks the columns with the highest missing-value share. The plot helps identify which fields would lose too many rows if the analysis simply deleted every incomplete record.

The report does not remove all incomplete rows. Instead, the numerical features used for modelling are filled with a median value and the categorical feature is filled with its most common value. The median is the middle observed value, so it is less affected by very large values

than the mean. This choice keeps more training examples while using a simple, explainable rule. The imputation rule is fitted on training data only, as explained later in the preprocessing section.

Leakage is different from missing data. Leakage occurs when an input contains information that would not be available at the time of a real prediction. A field such as an official final name or a false-positive flag can reveal the catalogue outcome directly. Including it would make a model appear strong because it reads the answer indirectly. For this reason, leakage-prone columns are excluded before the split and before training.

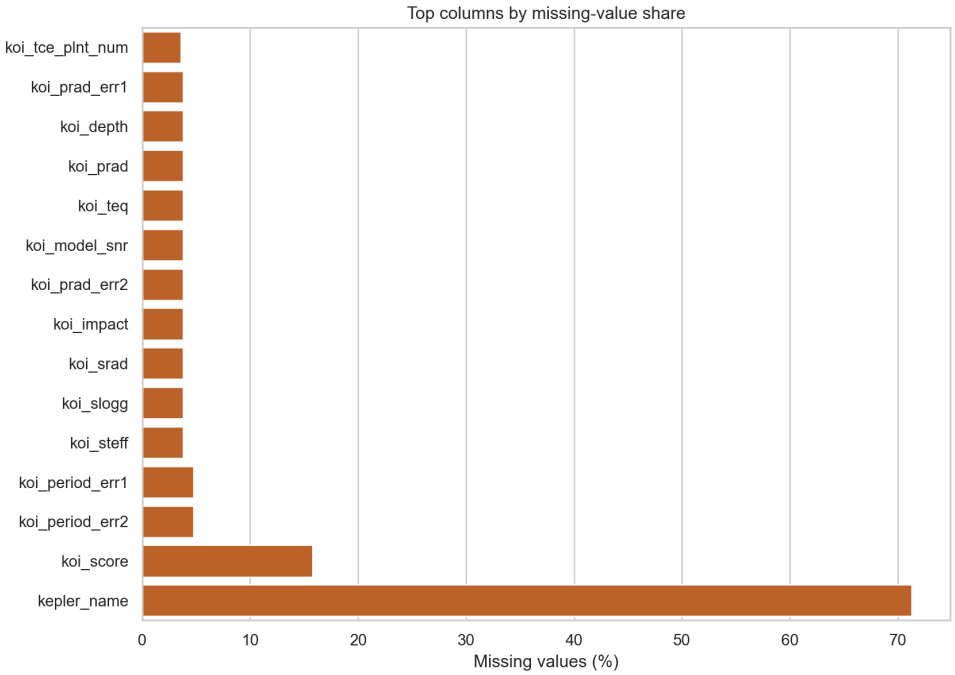



Figure 1.2: Highest missing-value shares among the selected KOI columns.

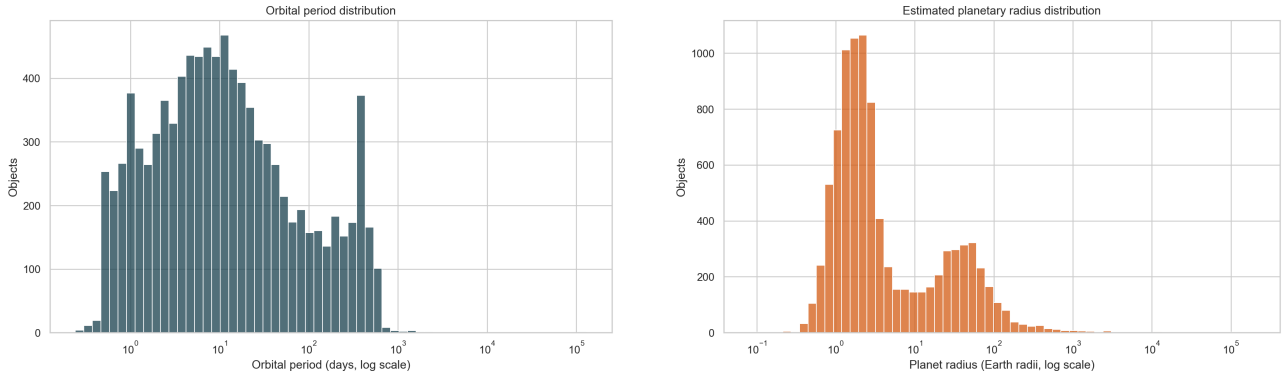
### 1.1.3 Numerical Distributions and Outliers

Numerical fields such as orbital period and estimated planet radius have long right tails. The plots use a log scale so that both small and large values are visible. Outliers are retained because unusual astronomical objects may still be valid observations.

The histograms in Figure 1.3 count objects in logarithmically spaced bins of orbital period and estimated planetary radius. They include only positive, non-missing values because the logarithmic axis cannot represent non-positive values. These filters apply only to the figures; they do not remove rows from the model data. The full values are still handled by the preprocessing pipeline.

Outliers are checked with the interquartile range (IQR) rule. For a feature, the first quartile $Q_1$ is the value below which 25% of observations lie, and the third quartile $Q_3$ is the value below which 75% lie. The IQR is $Q_3 - Q_1$. Values below $Q_1 - 1.5 \times IQR$ or above $Q_3 + 1.5 \times IQR$ are counted as possible outliers. Figure 1.6 reports this share for the selected numerical features. A

possible outlier is reviewed, not automatically deleted, because a rare astronomical measurement can still be a valid and informative object.




(a) Orbital period (b) Planet radius

Figure 1.3: Orbital period and estimated planetary radius. Bins and horizontal axes use a logarithmic scale; only positive, non-missing values are plotted.

The numerical EDA also includes host-star effective temperature and the remaining nine numerical model inputs. Figure 1.4 shows the temperature distribution and its median. Figure 1.5 shows those nine features. To make the central distributions easier to inspect, each panel displays values between that feature's first and 99th percentiles and reports the displayed and non-missing counts. Transit depth, insolation, and model signal-to-noise ratio additionally use logarithmic bins and show only positive values in that interval. These limits are for visualization only: they do not delete observations or change model preprocessing. Together with the period, radius, and stellar-temperature plots, this covers all 12 numerical model inputs.

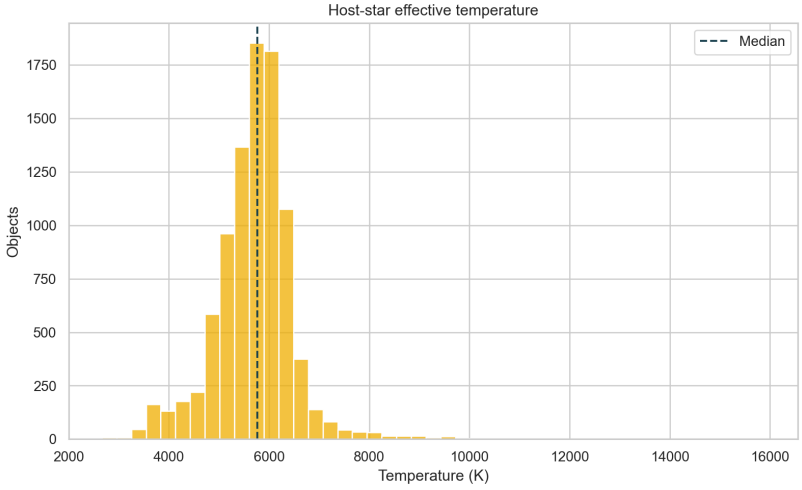



Figure 1.4: Host-star effective temperature among non-missing records; the dashed line marks the median. Counts refer to catalogue rows, not necessarily distinct host stars.

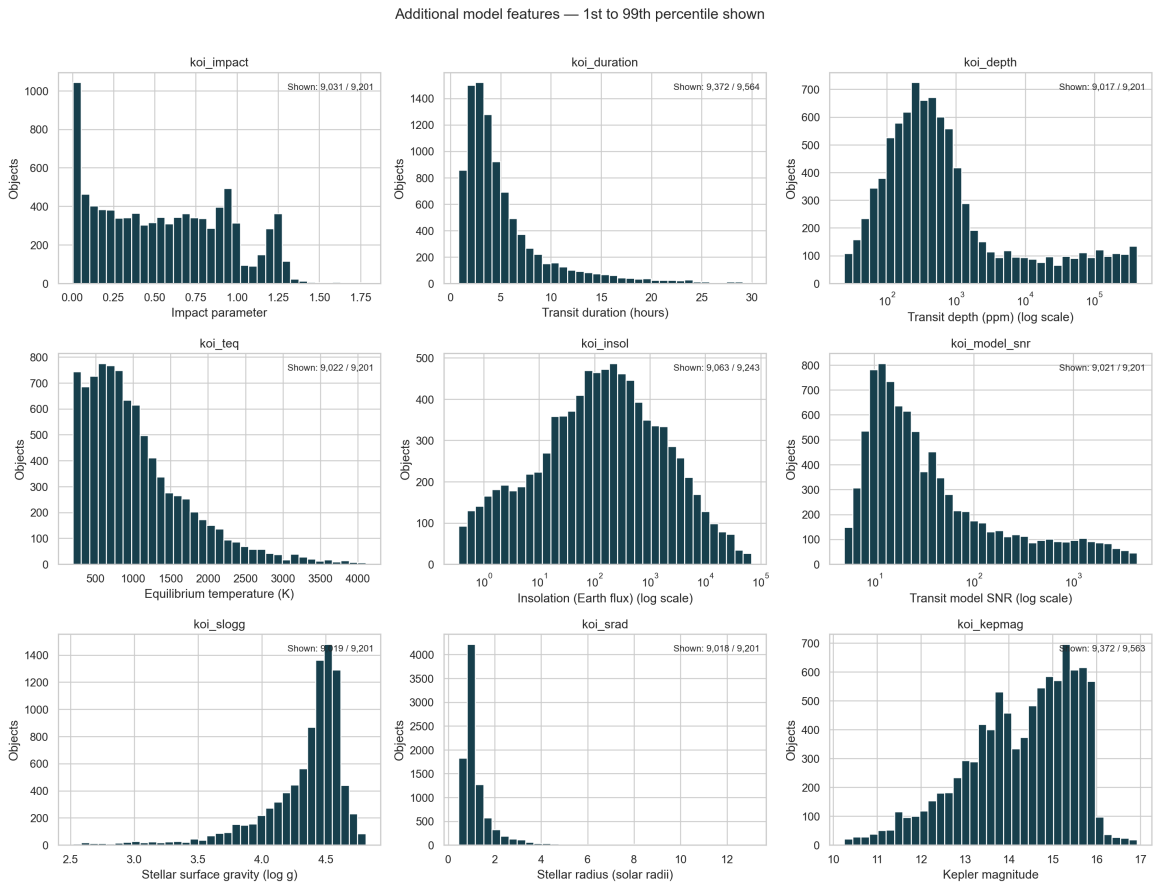



Figure 1.5: Distributions of the other nine numerical inputs. Each panel shows the first to 99th percentile of its feature; selected skewed features use logarithmic bins. The displayed and non-missing counts are annotated.

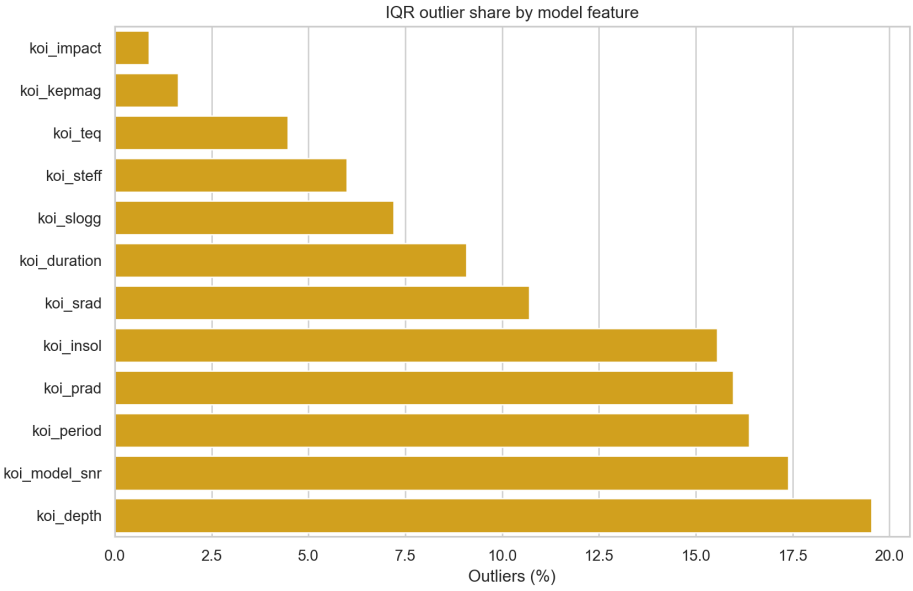
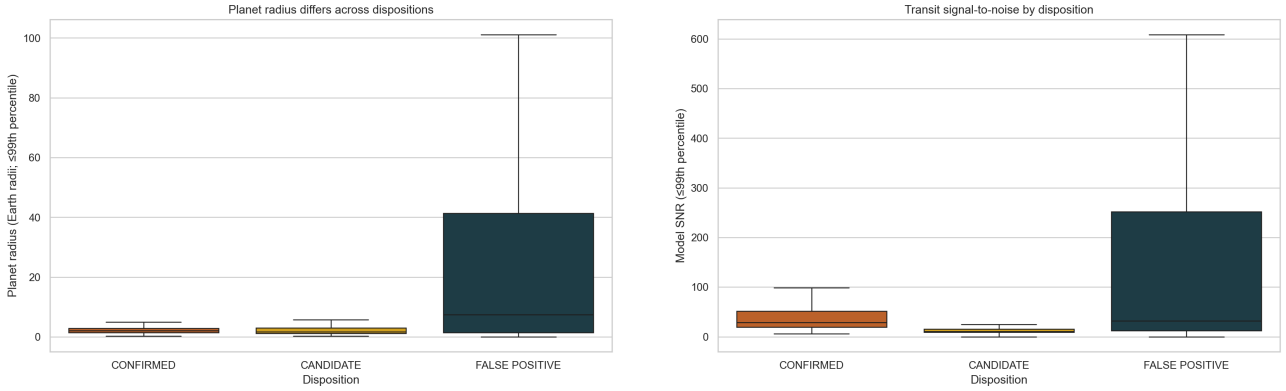



Figure 1.6: IQR outlier share for numerical model features. Outliers are audited but not deleted.

### 1.1.4 Feature Distributions by KOI Disposition

The class distributions overlap in every single feature. Figure 1.7 shows this for transit signal-to-noise ratio. A multivariable model is therefore needed.

The boxplots compare the median, central spread, and visible range of a feature for each target class. The most extreme one percent of values is hidden only in these comparison plots so that the central patterns can be read clearly; those rows are not removed from the dataset. The overlap shows that a simple rule based on radius alone would make many mistakes. The model needs several measurements together, because a useful pattern may be created by the combination of radius, period, signal strength, and other features.




(a) Planet radius by disposition

(b) Signal-to-noise ratio by disposition

Figure 1.7: Class-conditional distributions for two useful features. The overlap prevents a rule based on one column.

## 1.2 Preprocessing and Evaluation Design

### 1.2.1 Feature Preparation

The final model uses 12 numerical features and one categorical feature, `koi_tce_delivname`. Numerical missing values are imputed with the median. The categorical field is filled with the most common value and one-hot encoded. Logistic Regression uses standard scaling. Random Forest uses robust scaling in the same preprocessing pipeline.

The feature list is selected before modelling. The twelve numerical columns describe measurable properties of the signal and its host star. The categorical column identifies the TCE delivery catalogue. One-hot encoding turns each possible catalogue value into its own 0-or-1 indicator column. This lets a model use a category without treating its text name as a number with an artificial order.

The preprocessing pipeline has a fixed order. First, it receives only the selected columns. Second, it fills missing numerical values with the training median and missing category values with the training mode. Third, it scales numerical values and one-hot encodes the category. Finally, it sends the transformed feature matrix to the classifier. Keeping these steps in a

Scikit-learn pipeline prevents an accidental difference between training data and evaluation data.

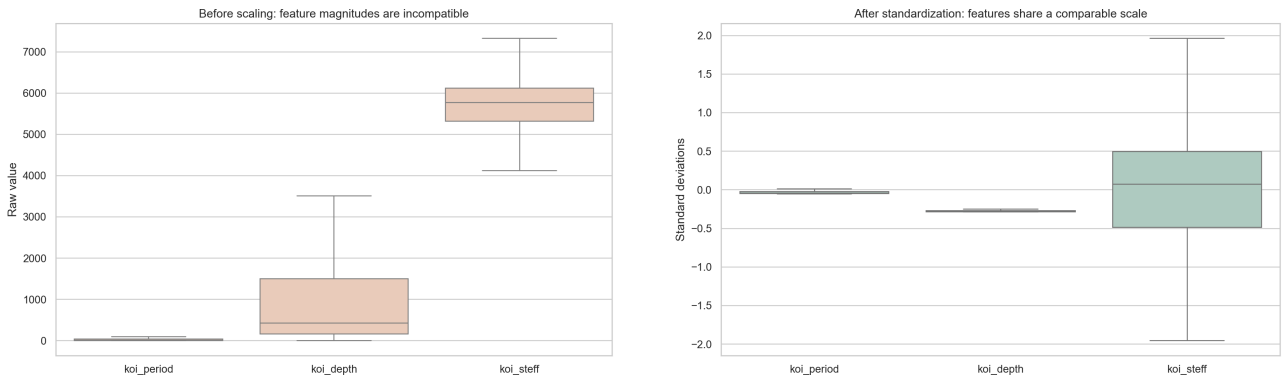
Standard scaling subtracts a feature mean and divides by its standard deviation, so features measured in days, radii, or kelvin have comparable sizes for Logistic Regression. Robust scaling uses the median and IQR instead, so a few extreme observations have less influence on the transformation. Tree models do not require scaling to make their splits, but the robust scaler is retained in the Random Forest pipeline to keep the numerical preparation stable and explicit. In both cases, the scaler learns its values from the training split only.

### 1.2.2 Group-Aware Train, Validation, and Test Split

The split is based on `kepid`, the host-star identifier. A GroupShuffleSplit separates data into 70% train, 15% validation, and 15% test. This prevents objects from the same star from appearing in more than one split.

The split is done in two steps. First, 30% of the host-star groups are held aside. Second, that held-aside part is divided equally into validation and test groups. The result is approximately 70% training, 15% validation, and 15% testing data. The word approximately is important because a group may contain more than one KOI object, so the exact row count is determined by groups rather than by individual rows.

This group-aware split is stricter than a random row split. If several signals from the same star appeared in both training and testing data, the model could benefit from star-specific information that it has already seen. Grouping by `kepid` makes the test closer to the real question: can the model classify signals from host stars that were not used for training?




(a) Before scaling

(b) After standardization

Figure 1.8: Scaling illustration for three numerical variables. The actual transforms are fitted using training data only.

## 1.3 Model Comparison and Final Evaluation

### 1.3.1 Baseline and Balanced Random Forest

Two models are compared on the validation split. Logistic Regression is the simple linear baseline. The final model is a Random Forest with 350 trees, balanced class weights, and a minimum leaf size of 2. Macro F1 is used to select the final model because it gives all three classes equal weight.

Logistic Regression is used first because it is a simple and interpretable baseline. For each class, it combines the prepared feature values with learned weights and produces a class score. A positive or negative weight changes the score, but the model can only form a linear boundary in the transformed feature space. If the relationship between measurements and the class is more complex, this baseline may be too simple.

Random Forest is an ensemble of decision trees. Each tree repeatedly asks a simple question, for example whether a measurement is below a learned threshold. The trees are trained with different samples and different subsets of features, then vote for the final class. Combining many trees can capture non-linear relationships and interactions between features. The minimum leaf size of 2 stops a tree from creating a final decision from only one row, and 350 trees make the voting result more stable.

The `class_weight` setting uses the balanced subsample option to give more importance to under-represented classes inside each tree sample. It does not create new KOI objects. Instead, an error on a less common class has a larger training cost, which encourages the forest to pay attention to CONFIRMED and CANDIDATE objects as well as the large FALSE POSITIVE class.

### 1.3.2 Validation Model Comparison

Table 1.2: Validation results for the tabular task.

| Model | Accuracy | Balanced accuracy | Macro F1 |
|---|---|---|---|
| Logistic Regression | 0.660 | 0.582 | 0.558 |
| Balanced Random Forest | **0.788** | **0.759** | **0.758** |

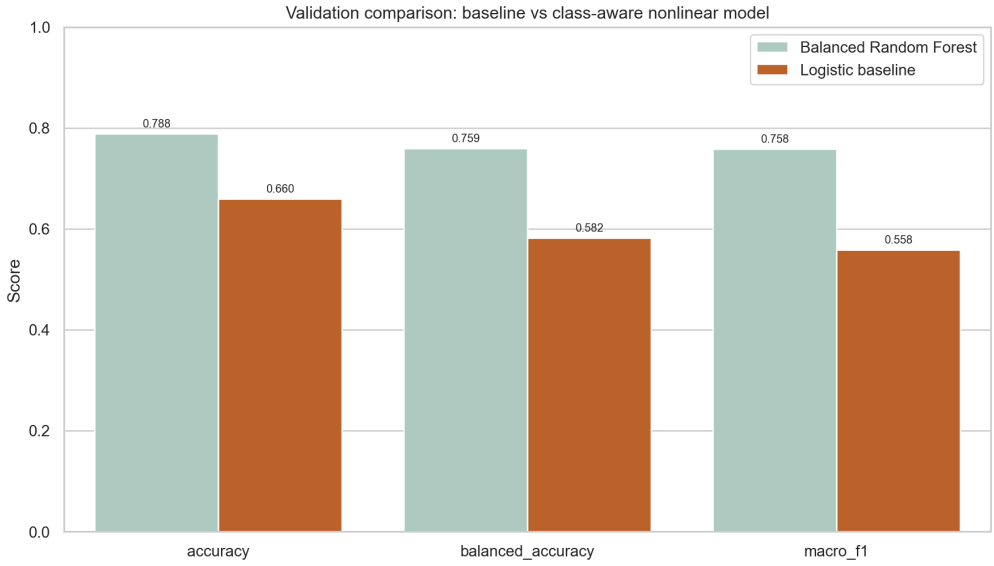



Figure 1.9: Validation comparison. The class-aware Random Forest improves all three metrics.

The validation table should be read as a model-selection result, not as the final claim. The Random Forest is higher on all three validation measures. In particular, its macro F1 is much higher than the Logistic Regression baseline, so the improvement is not only caused by making more correct predictions for the largest class. This is why the forest, rather than the baseline, is carried to the final test.

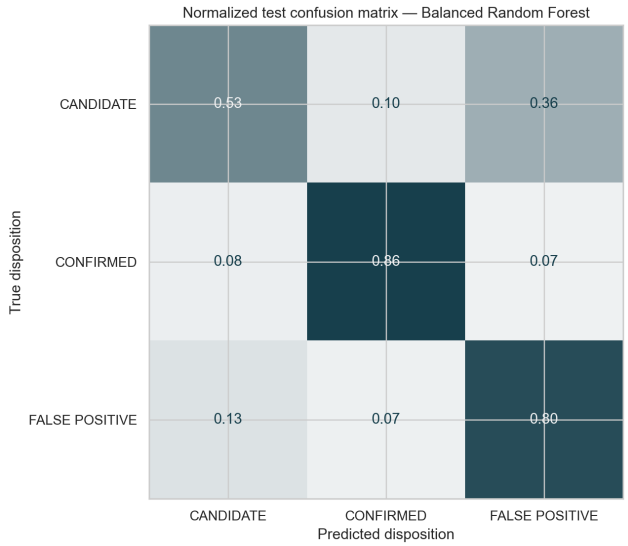
### 1.3.3 Final Test Results and Feature Importance

The selected model is evaluated once on the test split. It reaches 0.758 accuracy, 0.730 balanced accuracy, and 0.727 macro F1. The candidate class is the most difficult class, which is expected because it lies between confirmed planets and false positives.

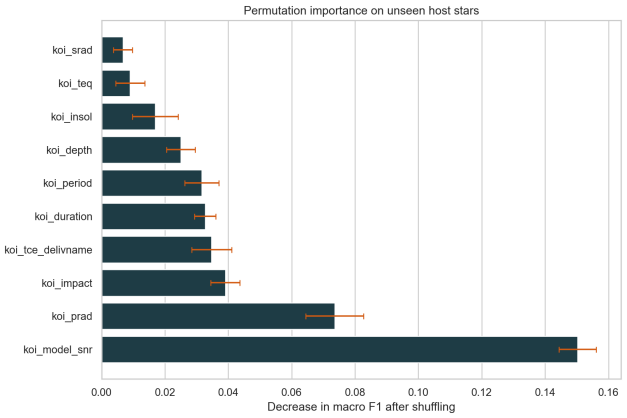
The test split is not used to choose hyperparameters or to change the feature list. It is used once after validation has selected the Random Forest. This separation is important: repeatedly looking at test results and changing the model would make the final score too optimistic.

The normalized confusion matrix in Figure 1.10 is read row by row. A row represents the true class and a column represents the predicted class. Values on the diagonal are correct predictions for that class. Values outside the diagonal show which classes are confused. The CANDIDATE row is harder because candidates can share measurement patterns with both confirmed objects and false positives. The matrix therefore explains an important limitation that a single accuracy number cannot show.

Permutation importance provides a second explanation of the selected model. For one feature at a time, the values in the test set are shuffled eight times while all other columns remain unchanged. If macro F1 decreases after that shuffle, the model was using that feature. A large decrease means the feature was useful to the current model on unseen host stars. This is a model-based importance score, not scientific proof that the feature causes the catalogue decision.




(a) Normalized test confusion matrix




(b) Permutation importance

Figure 1.10: Final test evaluation and feature importance for the Random Forest.

The three strongest features are `koi_model_snr`, `koi_prad`, and `koi_impact`. This does not prove a causal relationship. It only indicates that shuffling each feature reduces the model's macro F1 on unseen data.

The result is consistent with the earlier EDA: signal quality, estimated planetary radius, and transit geometry help separate the three catalogue labels. However, importance values should be interpreted carefully. Correlated measurements can share information, so one feature may look less important when another related feature is also present. The ranking is useful for explaining the trained classifier, not for replacing domain-specific astronomy analysis.

### 1.3.4 Key Findings and Limitations

**Tabular conclusion.** The dataset contains missing values, class imbalance, skewness, and outliers. The Random Forest is better than the linear baseline on the validation split. However, it is a catalogue classifier, not a system that can confirm a planet. Scientific confirmation still needs physical validation and follow-up observations.

# 2 Text Data: ViCTSD Vietnamese Comments

## 2.1 Exploratory Data Analysis

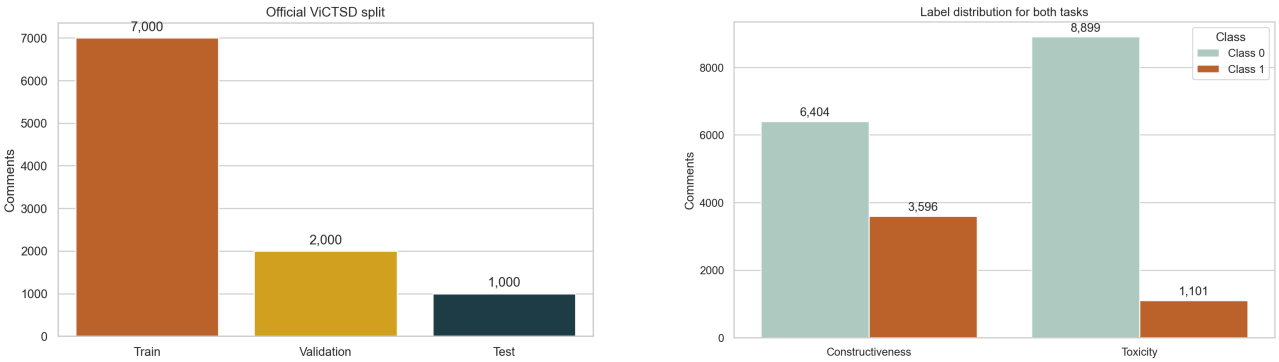
### 2.1.1 Dataset Size, Official Split, and Labels

The text task uses the official Vietnamese Constructive and Toxic Speech Detection dataset (ViCTSD)[3]. The source contains 10,000 Vietnamese social-media comments from ten news topics. Each comment has two binary labels: Toxicity and Constructiveness.

The two labels answer two different questions. The toxicity label asks whether a comment contains harmful or offensive language. The constructiveness label asks whether a comment contributes to a useful and respectful discussion. They are not treated as opposite labels in this report. A comment can be analysed by a toxicity model and by a constructiveness model separately, because the two targets describe different aspects of the same text.

Table 2.1: ViCTSD dataset summary.

| Item | Value |
|---|---|
| All comments | 10,000 |
| Topics | 10 |
| Official train split | 7,000 |
| Official validation split | 2,000 |
| Official test split | 1,000 |
| Toxic comments | 1,101 (11.01%) |
| Exact duplicate comments | 65 |




(a) Official split

(b) Label distribution

Figure 2.1: Official ViCTSD split and label distributions. Toxicity is the more imbalanced task.

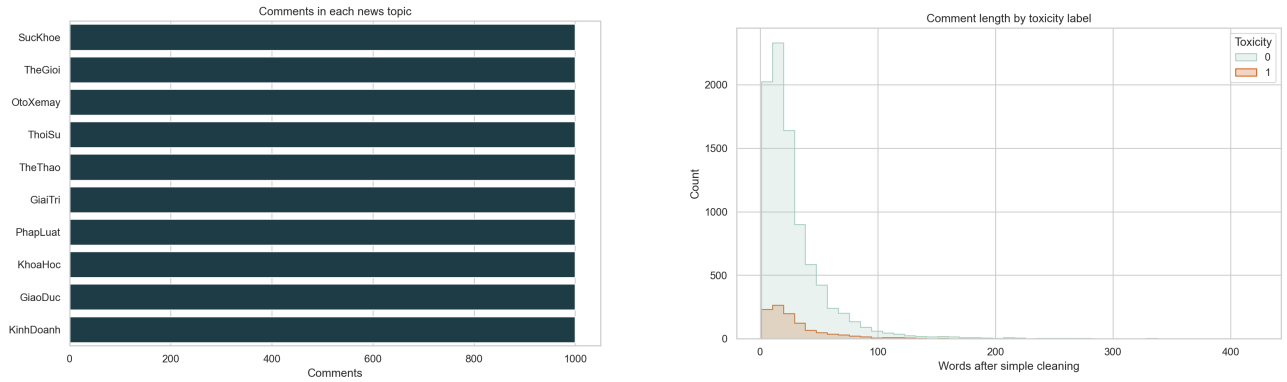
The official split is preserved as the starting point. The training split is used to learn

the text representation and model coefficients. The validation split is used to compare simple model choices. The test split is kept for one final evaluation after the choice is made. This order prevents the final score from influencing everyday modelling decisions.

### 2.1.2 Topic Balance and Comment Length

The topics are balanced: each topic contributes 1,000 comments. This is useful for exploration because one topic cannot dominate the basic count plots. Topic and article title are not used as model inputs. The first model only uses the comment text. This keeps the prediction question clear: the model should learn from the words in a comment, not from a topic name that may happen to have a different label pattern.

Topic balance does not mean that toxicity is equally distributed across topics. In the full descriptive dataset, 366 of 1,000 comments in the PhapLuat topic are labeled toxic (36.6%), compared with 13 of 1,000 in ThoiSu (1.3%). These are dataset-specific annotation rates, not causal claims or estimates for all comments in these categories.




(a) Comments per topic

(b) Comment length after cleaning

Figure 2.2: Text EDA. The data is balanced by topic, but comment length is right-skewed.

The comment-length histogram in Figure 2.2(b) counts whitespace-separated units after the notebook's simple text cleaning, grouped by toxicity label; it is not a character-count plot or a Vietnamese word-segmentation result. Across all 10,000 comments, this measure has mean 29.7, median 20, and maximum 422 units. Long comments are retained, and TF-IDF accepts comments of different lengths.

### 2.1.3 Vietnamese Word-Count Distribution

Whitespace units do not necessarily correspond to Vietnamese words: one word can contain several syllables separated by spaces. A separate descriptive analysis segments the raw comments with PyVi 0.1.1[4]. Each returned token containing at least one letter or digit counts as one segmented word; punctuation-only tokens and standalone emoji are excluded. This automatic segmentation is approximate, especially for informal spelling, slang, and mixed-language text, and is used only for EDA, not to train or select the reported models.

On the complete dataset of 10,000 comments, 408 comments contain exactly 10 segmented words and 238 contain exactly 20. Figure 2.3 displays exact counts from zero through 100 words; 206 longer comments are omitted from the figure but retained in the dataset and the downloadable count table. The longest segmented comment has 297 words. These counts describe comments rather than individual sentences.

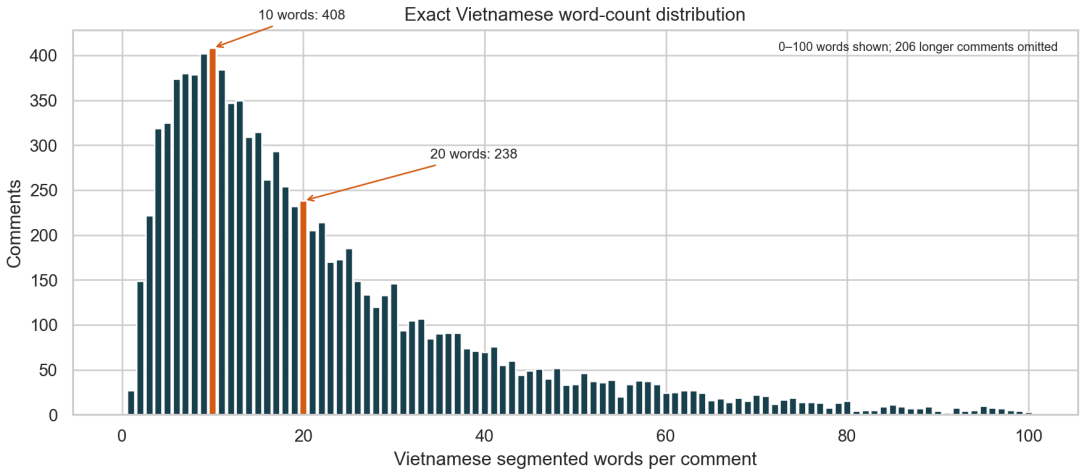



Figure 2.3: Exact distribution of PyVi-segmented word counts per comment across all official splits. The chart shows counts from 0 to 100; 206 comments beyond 100 words are outside the displayed range.

## 2.2 Text Preparation and Leakage Control

### 2.2.1 Simple Normalization

The cleaning process is intentionally simple: lowercase text, remove URLs, remove punctuation, and collapse repeated spaces. It is enough to make a first TF-IDF baseline reproducible. It is not intended to solve Vietnamese word segmentation or understand sarcasm.

The steps have direct purposes. Lowercasing treats two words that differ only by capital letters as the same word. Removing a URL avoids giving a unique web address an unnecessary feature. Removing punctuation and repeated spaces creates a consistent text form for counting words and phrases. Vietnamese letters and accents are kept. The model pipeline does not apply stemming, manual dictionaries, or a Vietnamese segmenter, because the goal is a small baseline whose operations can be read directly from the notebook. The separate PyVi analysis above is descriptive and does not alter model inputs.

Listing 1: Simple preprocessing used in the text notebook.

```
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"https?://\S+|www\.\S+", " ", text)
    text = re.sub(r"[^\w\s]", " ", text)
    return re.sub(r"\s+", " ", text).strip()
```

### 2.2.2 Duplicate and Split-Overlap Check

After this normalization, duplicate comments are checked across the official splits. Thirty-two validation rows and fifteen test rows have normalized text that already appears in training. These rows are excluded from evaluation to avoid direct text leakage. The final validation set has 1,968 rows and the final test set has 985 rows. Training remains at 7,000 rows.

An exact duplicate means that the original comment text is identical. A normalized duplicate may be written with different capital letters, punctuation, or extra spaces, but becomes the same after the simple cleaning function. The normalized check is stricter and is therefore the useful one for a TF-IDF model. Only evaluation rows that overlap with training are removed; the training split is not changed. Consequently, every validation and test score reported below refers to the filtered split of 1,968 or 985 comments, not to the unfiltered official count.

One normalized test comment in the checked data also occurs in the filtered validation set. It was not removed by the train-overlap filter, so the reported results still include it. Validation was used for model comparison; therefore the evaluation sets are not completely independent. A stricter follow-up should remove this overlap and recompute the metrics, figures, and report together.

**Why this check matters.** If the same sentence occurs in both training and testing, a bag-of-words model can receive credit for memorizing the sentence. Excluding overlap rows does not make the dataset perfect, but it avoids this direct and visible form of leakage.

## 2.3 Feature Representation

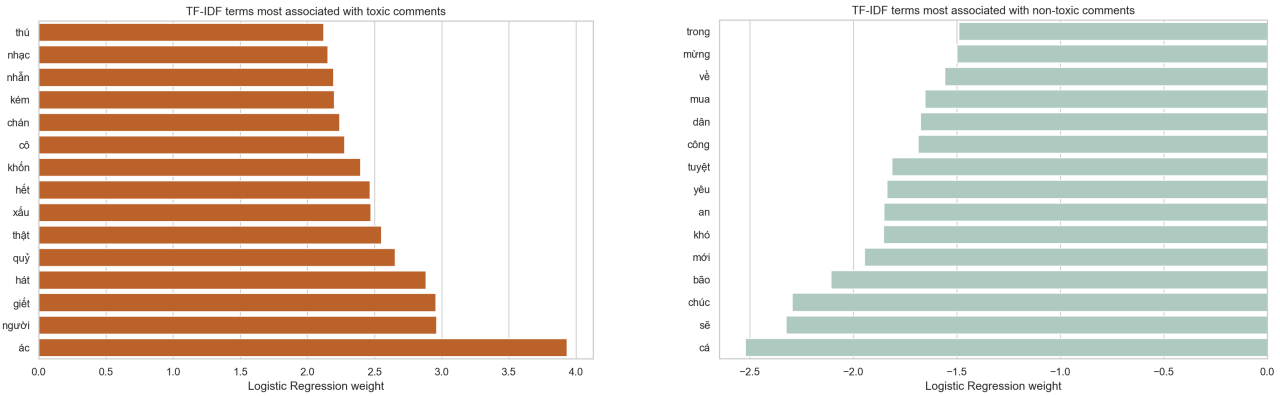
### 2.3.1 Word and Character TF-IDF

Two representations are tested. Word TF-IDF uses one-word and two-word phrases, with at most 8,000 features. Character TF-IDF uses character sequences of length three to five, with at most 12,000 features. Logistic Regression is used with balanced class weights for both representations.

TF-IDF changes each comment into a sparse row of numbers. A word or phrase receives a larger value when it is important in one comment but not common in all comments. Thus, a very frequent general word has less influence than a phrase that helps distinguish the two classes. The word model can learn separate signals from words and two-word phrases. The character model uses short sequences of characters instead; it may recognize related spellings or informal variants, but it can also be harder to interpret.

The vectorizers are fitted on training comments only. Terms that appear in validation or test data do not change the vocabulary or the TF-IDF weights learned from training. The word vectorizer ignores terms that occur in only one training comment and uses a maximum of 8,000 features. The character vectorizer uses the same simple rule with a maximum of 12,000 features. These limits keep the experiment small enough to run and inspect without hiding the

main idea behind a very large feature space.




(a) Higher toxicity weights

(b) Higher non-toxic weights

Figure 2.4: Word and phrase weights from the word-level Logistic Regression model. They are model signals, not fixed moderation rules.

Logistic Regression learns one coefficient for each TF-IDF feature. A feature with a higher positive coefficient pushes a comment towards the toxic class; a feature with a higher negative coefficient pushes it towards the non-toxic class. Figure 2.4 shows selected high-weight words and phrases. They must not be read as a fixed list of banned or safe expressions. The meaning of a phrase depends on its surrounding words, and a simple linear model sees only the features available in its input vector.

## 2.4 Toxicity Model Comparison

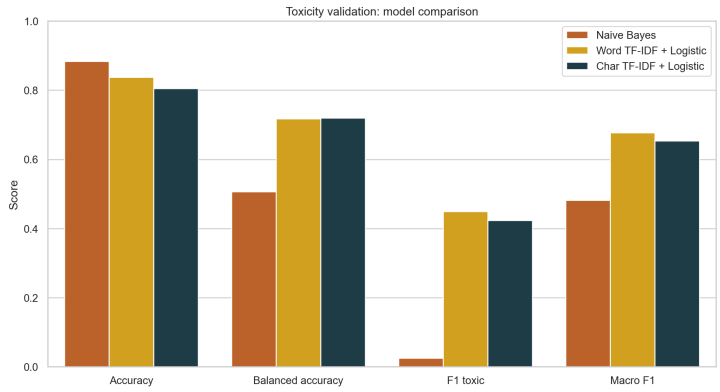
### 2.4.1 Validation Models and Selection

Three simple models are evaluated on the filtered validation set: Multinomial Naive Bayes with word TF-IDF, word TF-IDF with Logistic Regression, and character TF-IDF with Logistic Regression.

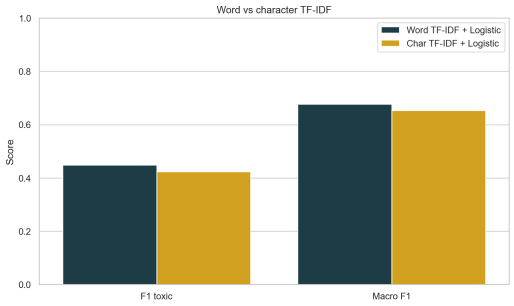
Multinomial Naive Bayes is a small baseline that estimates class evidence from feature frequencies and makes a simplifying independence assumption between features. Word TF-IDF plus Logistic Regression learns a direct linear decision from words and two-word phrases. Character TF-IDF plus Logistic Regression uses the same classifier but gives it character fragments instead of word features. For the two Logistic Regression models, balanced class weights increase the cost of mistakes on the less common toxic class. They do not duplicate comments; they only change how the training loss treats the two classes.

Table 2.2: Toxicity validation results.

| Model | Accuracy | Balanced acc. | F1 toxic | Macro F1 |
|---|---|---|---|---|
| Naive Bayes | **0.884** | 0.506 | 0.026 | 0.482 |
| Word TF-IDF + Logistic | 0.838 | 0.718 | **0.449** | **0.677** |
| Char TF-IDF + Logistic | 0.805 | **0.719** | 0.423 | 0.653 |




(a) Three-model comparison




(b) Word versus character TF-IDF

Figure 2.5: Validation model comparisons for the toxicity task.

Naive Bayes has the highest accuracy, but it almost never identifies toxic comments. Its toxic F1 is only 0.026. This is the direct effect of the minority class: a model can receive many correct non-toxic predictions while failing on the comments that need the most attention.

The selected model is word TF-IDF plus Logistic Regression because it has the best macro F1 and toxic F1 on validation. Macro F1 gives the toxic and non-toxic classes the same weight, while toxic F1 focuses specifically on the minority class. Its lower accuracy than Naive Bayes is therefore not a worse practical result. It is a more balanced result: the selected model accepts a small reduction in majority-class accuracy in order to recover many more toxic comments.

## 2.5 Final Test Results and Error Review

### 2.5.1 Test Metrics and Confusion Matrices

The selected toxicity model is tested once on the filtered test split. It achieves 0.842 accuracy, 0.732 balanced accuracy, 0.451 toxic F1, and 0.679 macro F1. The Constructiveness model uses the same word TF-IDF representation and reaches 0.800 macro F1 on the test split.

The confusion matrices in Figure 2.6 are normalized by true class. Each row represents comments with the same true label, and each cell shows the proportion predicted as each class. The diagonal cells are correct predictions. In the toxicity matrix, the toxic row is the most important row to inspect because it shows how many toxic comments are actually found. The same row-by-row reading applies to constructiveness.

Importantly, the two final macro F1 values should not be compared as if they measured one identical task. Toxicity and constructiveness use different target labels and have different class balances. The purpose of the table is to report how each selected simple model behaves on its own task, not to claim that one social-media property is inherently easier than the other.




(a) Toxicity

(b) Constructiveness

Figure 2.6: Normalized test confusion matrices for both text tasks.

### 2.5.2 Examples of Classification Errors

Table 2.3: Examples from the first toxicity errors. Text is shortened for readability.

| Topic | True | Pred. | Observation |
|---|---|---|---|
| OtoXemay | Toxic | Non-toxic | A short sarcastic statement is missed. |
| KhoaHoc | Toxic | Non-toxic | The context is stronger than individual words. |
| PhapLuat | Non-toxic | Toxic | Informal shorthand and strong wording create a false positive. |
| SucKhoe | Non-toxic | Toxic | A long context-dependent sentence is difficult for n-grams. |

A false negative occurs when a toxic comment is predicted as non-toxic. This is a missed harmful comment. A false positive occurs when a non-toxic comment is predicted as toxic. This can incorrectly flag a normal comment. The examples show why both error types occur. Sarcasm often has a meaning that is different from its individual words. Informal spelling, shorthand, and a sentence that depends on earlier discussion are also difficult for a model that only observes local words and phrases.

### 2.5.3 Key Findings and Limitations

**Text conclusion.** Word TF-IDF plus balanced Logistic Regression is a suitable simple baseline for this dataset. It is easy to inspect and it is better than the accuracy-only Naive Bayes baseline. Its limits are clear: sarcasm, shorthand, and long contextual comments still cause errors. The model can support review, for example by ordering comments for a moderator to inspect, but should not replace human judgement in moderation.

# 3 Image Data: Intel Image Classification

## 3.1 Task Objective and Dataset

This task classifies a natural scene image into one of six labels: *buildings*, *forest*, *glacier*, *mountain*, *sea*, and *street*. The selected source is the Intel Image Classification dataset on Kaggle[5]. It contains 17,034 labelled RGB JPEG images. Every original image is 150 by 150 pixels, which is above the course recommendation of a short side of 128 pixels. The dataset has 14,034 images in its source training folder and 3,000 images in a separate source test folder.

The image labels are supplied by the dataset. This project does not claim to discover a new scene category or to judge whether a source label is perfect. Its purpose is to build a clear and reproducible classroom classifier: inspect the images, define a fair split, compare two understandable pretrained approaches, and then measure the selected approach on images that were not used earlier.

**Task fit.** The dataset has more than 5,000 images and more than three target classes. It is therefore large enough for the Image-data task, while the six everyday scene classes remain understandable for a first image-classification project.

## 3.2 Dataset Structure and Image EDA

The downloaded dataset has two labelled folders: `seg_train` and `seg_test`. Each class has its own subfolder, so the parent folder name becomes the target label. The notebook uses `torchvision.datasets.ImageFolder` to read this simple structure. It does not read a hidden metadata column or a filename-derived target. The class order is alphabetical and is fixed as buildings, forest, glacier, mountain, sea, and street.

Table 3.1 shows the source distribution. The class counts are similar but not identical. Glacier has the most source-test images (553), while buildings has the fewest (437). For this reason, ordinary accuracy is reported together with balanced accuracy and macro F1. Macro F1 first computes an F1 score for each class and then gives every class the same weight. This makes a model that only succeeds on a common class less attractive.

Table 3.1: Intel Image Classification source folders and labels.

| Scene class | Source train | Source test | Total |
|---|---|---|---|
| Buildings | 2,191 | 437 | 2,628 |
| Forest | 2,271 | 474 | 2,745 |
| Glacier | 2,404 | 553 | 2,957 |
| Mountain | 2,512 | 525 | 3,037 |
| Sea | 2,274 | 510 | 2,784 |
| Street | 2,382 | 501 | 2,883 |
| **Total** | **14,034** | **3,000** | **17,034** |

Figure 3.1 gives one real image from each label. The examples already explain why this is a useful task: forest is visually distinctive because of repeated green texture, while glacier and mountain can share rocks, snow, and sky. Buildings and street can also share urban structures. These overlaps should appear again in the error analysis rather than being hidden behind a single score.

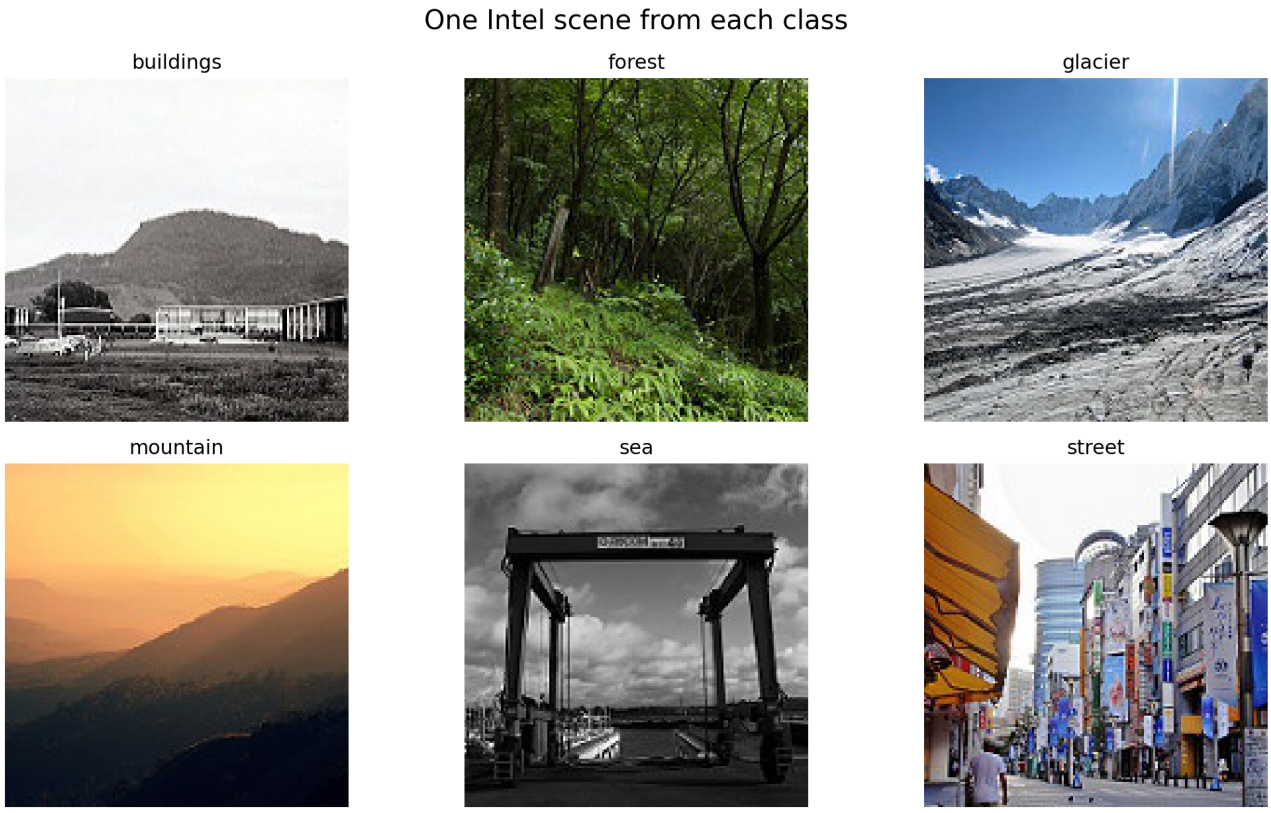
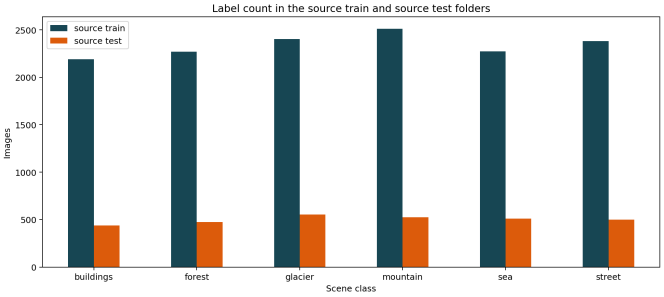



Figure 3.1: One source image from each Intel scene class.

Figure 3.2 contains two independent EDA plots. The first confirms the per-class counts.

The second uses mean pixel brightness as a small quality check. Brightness is not added as a separate model feature. It is only used to show that lighting, snow, clouds, and shadows vary across images, so a consistent image normalization step is necessary before using ImageNet-pretrained weights.




(a) Source class counts




(b) Brightness check by class

Figure 3.2: Simple image EDA. The brightness plot is an inspection result, not an input column.

## 3.3 Split, Quality Control, and Leakage Prevention

The source `seg_test` folder is reserved as the final test set from the beginning. It is never used to fit a model, choose an epoch, choose a transform, or compare the two approaches. The source `seg_train` folder is split with seed 42 and stratification: 80% becomes the fitting set and 20% becomes the validation set. Stratification keeps the six class proportions close to the original source distribution.

Table 3.2: Fair split used for the Intel image task.

| Split | Images | Use |
|---|---|---|
| Training | 11,227 | Fit the Logistic Regression classifier and MobileNetV3 head |
| Validation | 2,807 | Select the final approach using macro F1 |
| Source test | 3,000 | Evaluate the selected approach once at the end |

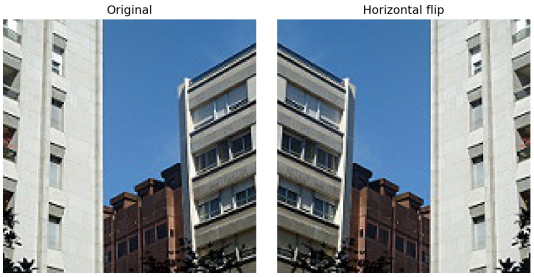
The code keeps the source folders separate and uses each image path in only one of these three roles. There is no outcome-like metadata or identifier used as an input: the input is only the RGB image. All 17,034 JPEG files loaded successfully during the executed run. This does not prove that a public dataset has no visually similar pictures, but it prevents the direct leakage that would occur if the provided test folder entered the split or model-selection step.

## 3.4 Preprocessing and Augmentation

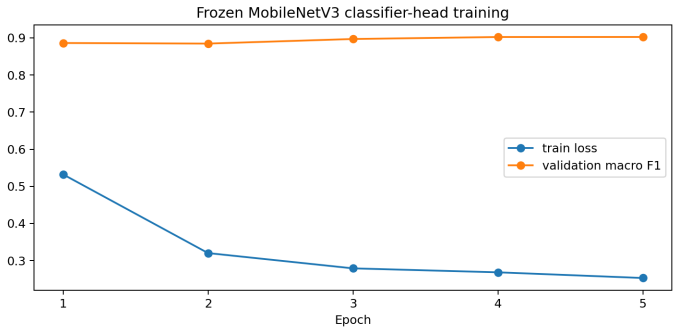
MobileNetV3-Small is pretrained on ImageNet. For feature extraction and evaluation, the notebook applies the preprocessing supplied with the pretrained weights: resize, centre crop,

tensor conversion, and ImageNet channel normalization. The original data are 150 by 150 pixels; the network receives its standard 224 by 224 view after resizing. This makes the numerical pixel range compatible with the pretrained network.

The training view also uses one simple augmentation: a random horizontal flip. Mirroring a forest, a mountain, or a street does not change its scene label, but it prevents the new classifier head from relying too strongly on a left-right layout. Validation and test images never receive the random flip, so their predictions are repeatable. Figure 3.3 shows one augmentation example and the five-epoch learning curve.




(a) Random horizontal flip




(b) Frozen-head training history

Figure 3.3: The only random augmentation and the five-epoch training history.

## 3.5 Model Comparison

Both candidate models reuse the same pretrained MobileNetV3-Small visual backbone. This meets the deep-learning requirement without asking a beginner to train a convolutional network from scratch. The first model removes the final classifier, so each image becomes a 576-value pretrained feature vector. A balanced Logistic Regression model is then fitted to these vectors. This is a useful simple baseline: the deep network describes visual patterns, and a linear classifier makes the final class decision.

The second model freezes the trainable parameters of the MobileNetV3 feature layers and replaces its final output layer with a six-class classifier. The classifier head is updated for five epochs with Adam, learning rate 0.0003, and class-weighted cross-entropy. The backbone's BatchNorm running statistics can still change while the model is in training mode, even though its parameters are frozen. This distinction matters when describing what remains fixed.

Table 3.3 compares the models only on validation images. The pretrained-feature Logistic Regression model has the higher validation macro F1 (0.906 versus 0.902), so it is selected before the source-test labels are evaluated. The difference is small, but the selection rule is explicit and does not use the test score.

Table 3.3: Validation comparison. Selection is based on macro F1.

| Model | Accuracy | Balanced acc. | Macro F1 |
|---|---|---|---|
| Pretrained features + Logistic Regression | **0.904** | **0.906** | **0.906** |
| Frozen MobileNetV3 head | 0.900 | 0.902 | 0.902 |

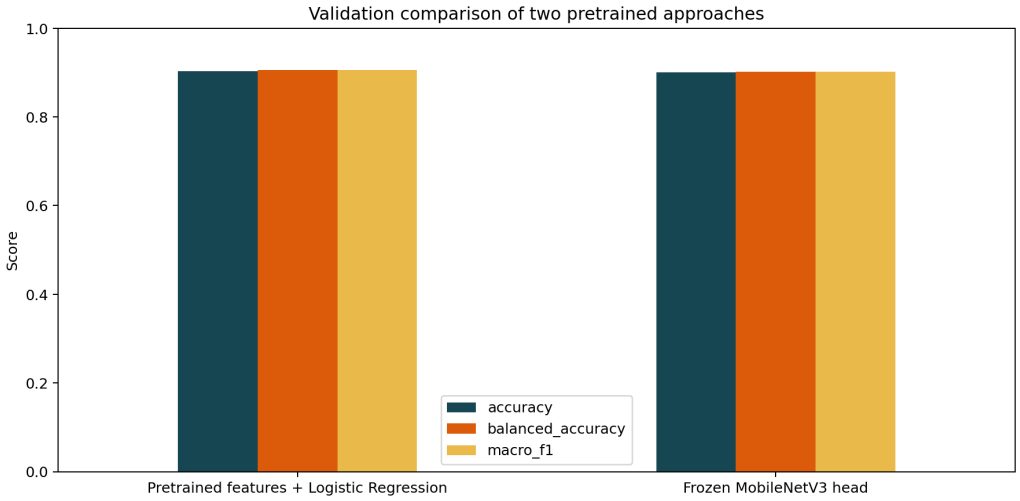



Figure 3.4: Validation scores for the two pretrained approaches.

## 3.6 Final Test Evaluation and Error Review

The selected pretrained-feature Logistic Regression model is evaluated once on the untouched source test folder. It obtains accuracy 0.914, balanced accuracy 0.916, and macro F1 0.916. The close values show that the result is not caused only by one large class. Forest is the easiest class (F1 0.994), while glacier and mountain are harder (F1 0.860 and 0.863). This matches the visual overlap seen in the example images.

Table 3.4: Per-class result of the selected model on the 3,000-image source test set.

| Class | Precision | Recall | F1 |
|---|---|---|---|
| Buildings | 0.903 | 0.920 | 0.912 |
| Forest | 0.996 | 0.992 | 0.994 |
| Glacier | 0.863 | 0.857 | 0.860 |
| Mountain | 0.867 | 0.859 | 0.863 |
| Sea | 0.938 | 0.949 | 0.943 |
| Street | 0.928 | 0.922 | 0.925 |
| **Macro average** | **0.916** | **0.916** | **0.916** |

Figure 3.5 is normalized row by row. A diagonal cell is the recall for one true class. The darker diagonal cells confirm good recognition of forest and sea. The main visible confusion is between glacier and mountain, which is expected because both can contain rock, snow, cloud, and a distant horizon.

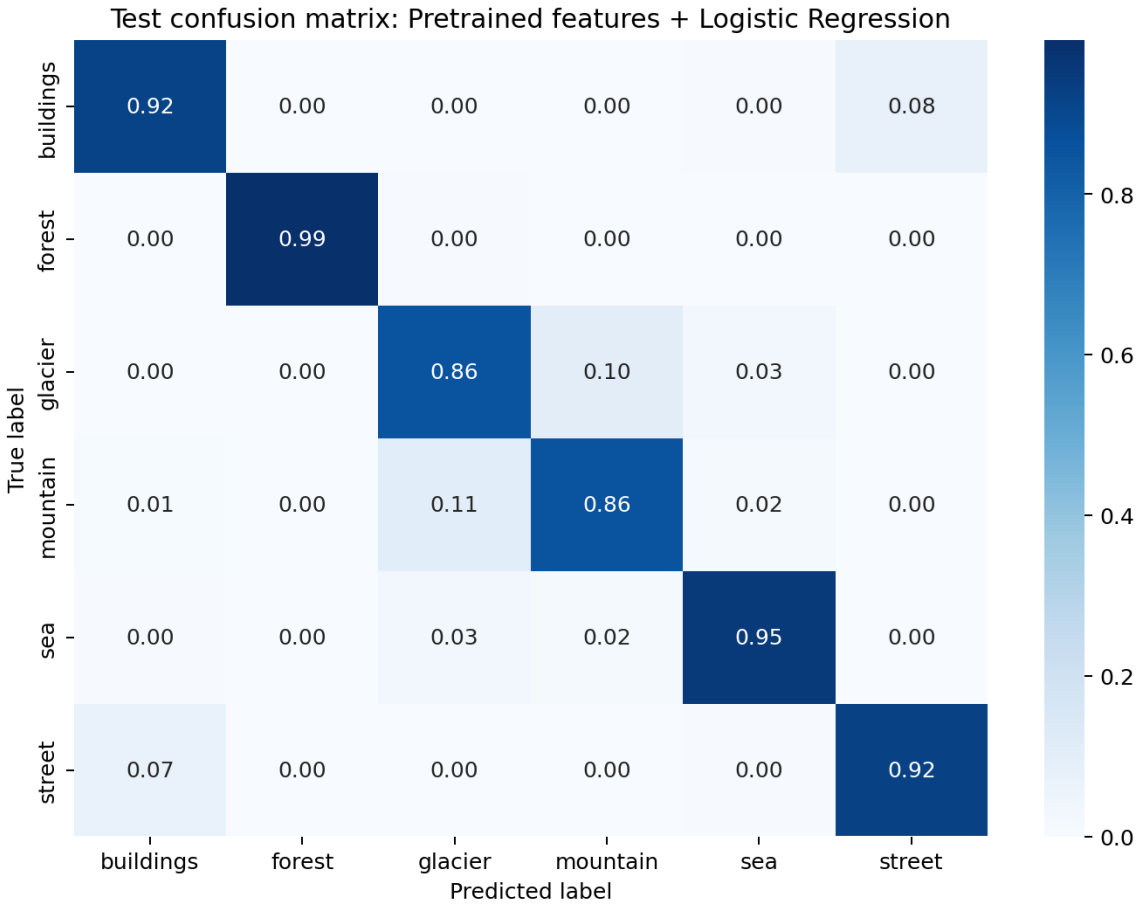



Figure 3.5: Normalized source-test confusion matrix for the selected model.

There are 257 wrong predictions among the 3,000 test images. The two most frequent directional errors are glacier predicted as mountain (58 images) and mountain predicted as glacier (58 images). Street predicted as buildings (35 images) and buildings predicted as street (33 images) are the next important pair. Figure 3.6 shows twelve concrete mistakes. These examples are more useful than claiming that a 0.916 macro F1 is perfect: scene boundaries can be visually ambiguous, and a model can fail on unusual lighting, distant objects, or mixed scenery.

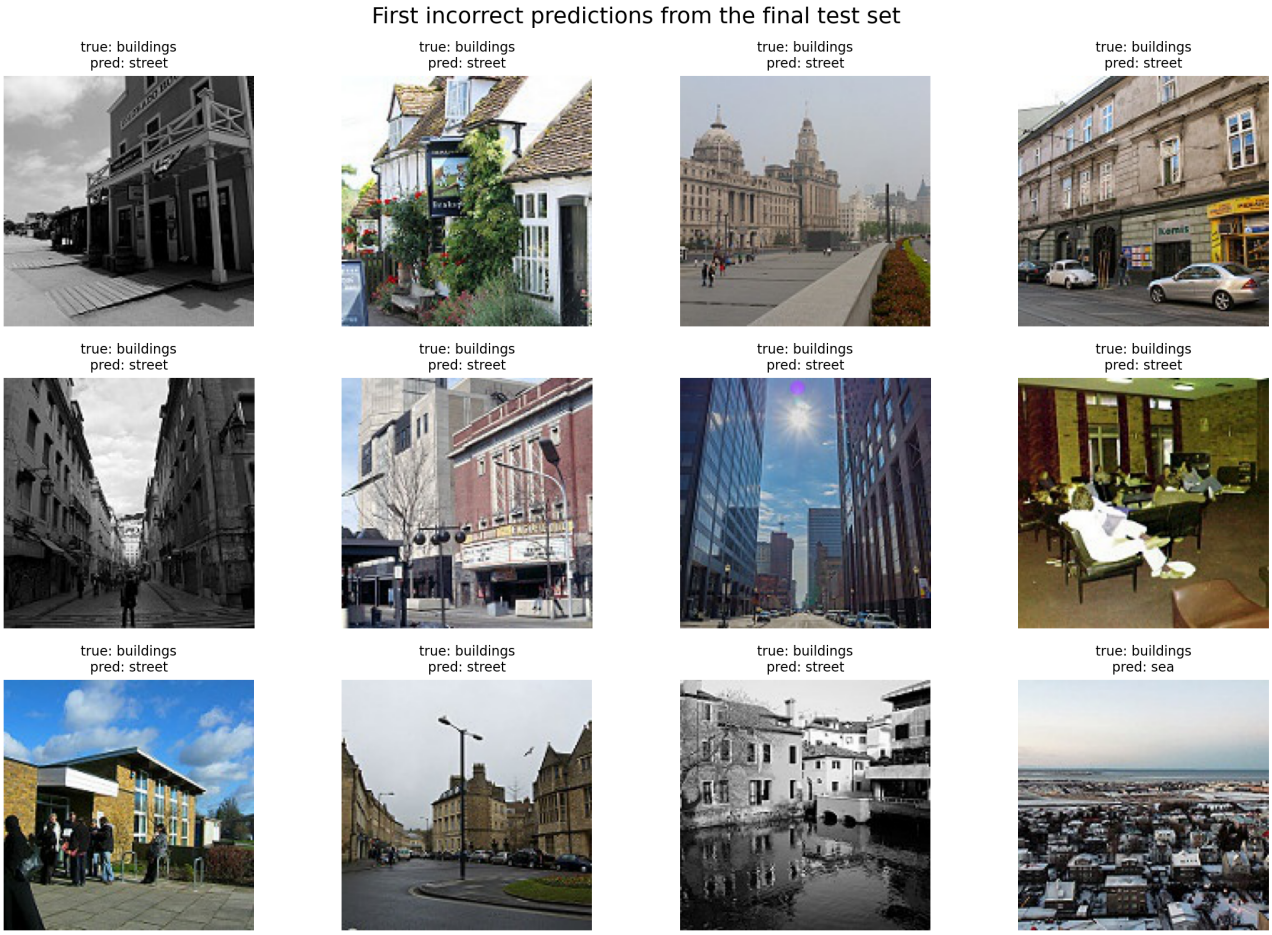



Figure 3.6: Twelve incorrect predictions from the selected model on the untouched source test set.

**Image conclusion.** Pretrained MobileNetV3-Small features plus balanced Logistic Regression are a strong, simple, and reproducible baseline for Intel scene classification. The result meets the assignment workflow because EDA, augmentation, a pretrained deep representation, a validation comparison, and a final untouched test evaluation are all present. It is still a classroom model: it can confuse visually related scenes and should not be treated as a general-purpose real-world scene understanding system.

# 4 Overall Conclusion

The three tasks show the same lesson: EDA changes model decisions. The same general workflow is used in every task, but the details must be adapted to the data type. For KOI records, one row contains measured numerical and categorical properties of one astronomical object. The key risks are missing measurements, skewed values, and fields that reveal the final catalogue decision. For comments, one row contains a variable-length sentence. The key risks are repeated normalized text, an imbalanced toxic class, and wording that has a meaning only in context. For Intel scenes, one observation is a pixel image. The key risks are class imbalance, brightness and background variation, train–test leakage, and visually similar scenes such as glacier and mountain.

The analysis framework is therefore simple but explicit. First, inspect the target, size, missing values, duplicates, and distributions. Second, define what information is allowed as input and remove direct leakage. Third, prepare the input using rules learned from training data only. Fourth, compare a small baseline with one stronger but still understandable model on validation data. Finally, use the test set once and inspect errors instead of relying on one headline score. These steps make the result easier to check and reduce the chance that a score is improved by an unfair shortcut.

## 4.1 Summary of Final Results

The final results are summarized below. Accuracy counts all correct predictions. Balanced accuracy checks whether every class is recognized fairly. Macro F1 balances precision and recall for each class before averaging them. For this reason, macro F1 is the most useful single comparison measure in this report, especially for the imbalanced toxicity task, the three-class KOI task, and the 10-class image task.

Table 4.1: Final selected-model results.

| **Task** | **Selected model** | **Accuracy** | **Balanced acc.** | **Macro F1** |
|---|---|---|---|---|
| Kepler KOI | Balanced Random Forest | 0.758 | 0.730 | 0.727 |
| ViCTSD Toxicity | Word TF-IDF + Logistic | 0.842 | 0.732 | 0.679 |
| ViCTSD Constructiveness | Word TF-IDF + Logistic | 0.807 | 0.812 | 0.800 |
| Intel Image Classification | Pretrained features + Logistic | 0.914 | 0.916 | 0.916 |

## 4.2 Limitations and Future Work

The code is deliberately compact and beginner-friendly. This makes the workflow easier to reproduce, but it also creates clear limits. The KOI model uses a fixed set of catalogue variables and a simple group split; it cannot replace scientific confirmation from physical follow-

up observations. The text model has no sentence-level understanding of sarcasm, conversation history, or implied meaning. Its predictions should be treated as model suggestions, not final moderation decisions. The Intel scene model observes only image pixels after resizing to 150 by 150. It can confuse visually related scene categories (such as glacier and mountain, or street and buildings) and cannot replace human ground truth.

Future work may add carefully designed KOI features, compare different group-aware resampling seeds, or calibrate predicted probabilities. For text, later experiments may incorporate Vietnamese word segmentation into model inputs, evaluate a pretrained Vietnamese language model, or review important error groups manually. For images, later work may carefully fine-tune more MobileNetV3 layers, test additional valid augmentations, or review ambiguous scene error groups. Each improvement would need to repeat the same leakage checks and keep the test split independent. These are possible next experiments, not results claimed by the current report.

# Reproducibility

The following project files can reproduce the analysis. A simple order is: install the notebook requirements, place the data files in the stated data folder, run the tabular notebook from top to bottom, then run the text notebook, and finally run the image notebook. The notebooks create the tables and figures used by the report. Re-running a model with the documented random seed makes the split and the main reported results repeatable in the same software environment.

- `notebooks/tabular_koi_eda.ipynb` for the Kepler KOI task.
- `notebooks/text_victsd_eda.ipynb` for the ViCTSD task.
- `notebooks/image_intel_eda.ipynb` for the Intel Image Classification task.
- `data/koi_cumulative.csv`, the three `data/victsd_*.csv` files, and public source folders downloaded by the image notebook.
- `reports/figures/` for the generated figures.
- `reports/` word-count generation script and distribution CSV for the separate PyVi word-count analysis; install PyVi 0.1.1 to regenerate it.
- The Intel JSON summary and test-prediction CSV in `reports/` for the selected image-model result.

# References

[1] Le Thanh Sach. *Programming Foundation for Data Analytics and Visualization: Course Project Assignment, Version 4.0*. Ho Chi Minh City University of Technology, Semester 261, September 2026.

[2] NASA Exoplanet Archive. *Kepler Objects of Interest Cumulative Table*. Available at: https://exoplanetarchive.ipac.caltech.edu/docs/Kepler_KOI_docs.html. Accessed October 2026.

[3] tarudesu. *ViCTSD: Vietnamese Constructive and Toxic Speech Detection*. GitHub repository. Available at: https://github.com/tarudesu/ViCTSD. Accessed October 2026.

[4] Trung Tran. *PyVi: Python Vietnamese Core NLP Toolkit*. Available at: https://github.com/trungtv/pyvi. Version 0.1.1 used for word segmentation.

[5] Puneet6060. *Intel Image Classification*. Kaggle dataset. Available at: https://www.kaggle.com/datasets/puneet6060/intel-image-classification. Accessed October 2026.